TECHNICAL REPORT · OCTOBER 2026

# Hisab-Kitab AI

A Hinglish bookkeeping assistant for small local businesses — system design and evaluation on unseen data, with a brick and timber salvage business as the test bed

**Sahil Verma**
M.Sc. Mathematics, Indian Institute of Technology Kharagpur

sahilverma.24@kgpian.iitkgp.ac.in · sahilvermahr16@gmail.com
github.com/sahilvermahr16-cmyk/hisab-kitab-ai
linkedin.com/in/sahil-verma-990001319

**75.3%**
of entries strictly correct (125/166, 95% CI 68.2–81.2%)
135 unseen statements · 103 fully right

**9.6%**
silent-error rate on unseen data
down from 18.5% in holdout round 1

**695**
locked test statements
425 development + 270 holdout

Model: gpt-4o-mini (temperature 0 in all accounting steps) · Framework: LangGraph · Database: PostgreSQL
Full per-statement results: *Hisab-Kitab_Testing_Report_v2.xlsx* (companion workbook)

# Executive summary

**Problem.** Most small local businesses in India (kirana and hardware shops, building-material traders, dairies, contractors) still keep accounts in a paper diary, or not at all: the owner has no time to write daily, and accounting software expects forms, English menus and training. Hisab-Kitab AI lets the owner simply say what happened. It was built and tested with my father's brick and timber salvage business in Bhiwani, Haryana, where the day is a stream of short Hinglish notes such as *"Mange Ram ko makaan ke 50000 diye"* ("Paid Mange Ram 50,000 for the house") or *"Dalbir 300 eent le gaya"* ("Dalbir took 300 bricks"). Each note can hide a wage, a sale, a stock movement or a family expense, often several at once, with number words (*dedh, saade, sava* = one and a half, plus a half, plus a quarter) and relative dates (*kal, parso* = yesterday, the day before).

**System.** Hisab-Kitab AI is a LangGraph agent that turns such notes into double-checked database entries across 18 accounting tools (wages, attendance, sales, supplier deliveries, purchases, expenses, stock moves, adjustments, family withdrawals, opening balances). An LLM (gpt-4o-mini) does the language work at eight steps of the accounting flow; after each step a layer of deterministic code rules ("code nets") checks and repairs the output. The user confirms every entry before it is written, and the system asks when a role, amount or place is unclear.

**Evaluation.** Test statements were locked before each run and scored strictly: an entry counts as correct only if the tool and every key field (person, amount, date, item, quantity, rate, place) are right. The most important number for an accounting tool is the **silent-error rate**: a wrong entry the user is not warned about.

- **Development (425 statements, 10 bundles).** Strict accuracy rose from 42–76% before fixes to 85–100% after fixes. These numbers are optimistic, because the fixes were made while looking at the same statements.
- **Holdout round 1 (135 unseen statements, 178 scored entries).** 62.9% of entries strictly correct, 18.5% silent errors, 90 of 135 statements fully right. The development numbers did not transfer; its failures led to seven groups of fixes.
- **Holdout round 2, final (135 new unseen statements, run once, no fixes after it).** The statements gave 166 scored entries: **75.3% of entries strictly correct (95% CI 68.2–81.2%)**, **9.6% silent errors**, and **103 of 135 statements (76.3%) fully right**. For the entry bundles the two intervals do not overlap, so the gain is unlikely to be chance alone.

**Main finding.** Each fix that taught the system a specific word improved the development score but transferred only partly to new data. The remaining errors come from a few general causes: a short name matched inside a longer name, "*le gaya*" ("took") read as goods when it meant money, an amount separated from its owner, and a place read as a person. The next version should teach decision principles rather than word lists, give the grouping step the full original statement, and ask instead of dropping an amount it cannot place (Section 8).

**Limits.** The test statements follow the way my father actually speaks; they were drafted with the help of an AI assistant and checked and locked by me, but they are not yet recordings from live use. All testing was on one business; the core design is general, but transfer to a second business has not been measured. Only Hinglish was tested. With 10–33 rows per bundle, per-bundle numbers have wide intervals.

# Contents

1. Introduction
2. Background and related work
3. System design
4. Evaluation methodology
5. Results
6. Error analysis
7. Limitations
8. Conclusion and future work

References

Appendix A — Example test statements

Appendix B — Fix log summary

Appendix C — Proof files and test dates

Appendix D — Scoring rules in detail

Appendix E — Sample report output

# 1. Introduction

## 1.1 The problem: books that never get written

Millions of small local businesses in India run on a paper diary: a kirana shop, a hardware store, a building-material trader, a dairy, a small contractor. The owner is busy all day, so the diary is written late, partly, or not at all, and dues and stock drift. Ready-made accounting software does not fit them: it needs forms, English menus, a computer and training. What the owner can do is say one sentence in his own language. Hisab-Kitab AI turns that sentence into correct, confirmed accounting entries.

## 1.2 The test bed: a brick and timber salvage business

The system was built and tested with one real business, my father's brick and timber salvage business in Bhiwani, Haryana. It is a demanding first case because it has every kind of entry: daily wages and attendance, sales on credit, supplier deliveries, purchases of whole buildings, expenses, stock moving between places, and family cash taken from the till. The business has three kinds of places: **sites** (old buildings being demolished, such as "Kalanaur" or "Dhani wali site"), a **godown** (*gudam*) where salvage is stored, and a **shop** (*dukaan*) where it is sold. Around them are employees paid daily wages, customers who buy on credit, brick kilns and timber merchants who supply goods, owners who sell old buildings, and family members who take cash from the till (*galla*).

## 1.3 Why this is hard

- **One sentence, several entries.** *"Suresh Traders 5 shahteer le gaye 10000 ke, 4000 de gaye"* ("Suresh Traders took 5 wooden roof beams worth 10,000 and paid 4,000") is a sale *and* a payment.
- **The same verb means different things.** *"Dalbir 300 eent le gaya"* ("Dalbir took 300 bricks") is a sale of bricks; *"Naresh 600 le gaya"* ("Naresh took 600") is money given to an employee.
- **Roles are not stated.** Whether "Ram" is an employee, a customer or a seller must come from the database or from a question.
- **Hinglish numbers and dates.** *dedh sau* = 150, *sava sau* = 125, *saade paanch* = 5.5, *kal* = yesterday, but "*Kal*anaur" is the name of a village.
- **Mistakes are expensive.** A wrong entry that nobody notices corrupts every later balance. It is better to ask than to guess.

## 1.4 Goal and contributions

The goal was a working assistant that a small business owner can use, first deployed for my father, and built and tested with the discipline of a research project. This report contributes:

1. a hybrid LLM + rule pipeline for Hinglish bookkeeping with human confirmation (Section 3);
2. a test method with locked statements, strict field-level scoring, a silent-error metric, confidence intervals and two holdout rounds (Section 4);
3. an honest measurement of how much development-set fixes transfer to unseen data (Section 5), and an analysis of the errors that remain (Section 6).

## 2. Background and related work

**LLMs for information extraction.** Large language models can turn free text into structured records when asked for a fixed output schema. Here every LLM step returns a Pydantic object, so outputs are validated rather than parsed from free text. LLMs are still not fully deterministic or reliable on rare wording, which is why every step is followed by code checks.

**Code-switched language.** Hinglish mixes Hindi and English words, usually in Latin script, with no standard spelling (*gudam / godam / gudm*, all meaning "godown"). Benchmarks such as GLUECoS [3] show that code-switched text is harder for language models than either language alone.

**Agent frameworks.** LangGraph [4] models an agent as a graph of steps with shared state. It supports parallel branches, sub-graphs, and pausing for a human answer (`interrupt`) with a persistent checkpoint. These features suit a bookkeeping flow that must stop and ask.

**Behavioural testing.** CheckList [5] argues that NLP systems should be tested by capability with many small targeted cases, not only by one aggregate score. The test bundles in this project follow that idea: each bundle targets one business area, and each category inside it targets a kind of difficulty.

**Reporting and leakage.** Model cards [6] and later evaluation-report guides ask for the exact system tested, the protocol, uncertainty and limits. Kapoor and Narayanan [7] show how leakage between development and test data inflates results. This is the reason for the leakage check and the two holdout rounds used here.

**Uncertainty.** All accuracy intervals use the Wilson score interval [1], which behaves well for small samples and for rates near 0% or 100%.

# 3. System design

## 3.1 Overview

Every message first goes to a router (one LLM call), which sends it down one of three paths. **General chat** handles small talk (*“Aaj bahut thand hai”* (“It is very cold today”)) with a normal reply and writes nothing. **Accounting** handles anything that changes the books (a wage, a sale, a payment, stock moving) and is the main pipeline described below. **Report** handles requests to see the books (*“Sharma ji ka kitna baaki hai”* (“How much does Sharma ji still owe?”)): an LLM reads what is asked (whose account, which dates), the system asks only for what is missing, and the numbers themselves are computed by SQL and Python, never by the LLM, and returned as a PDF.

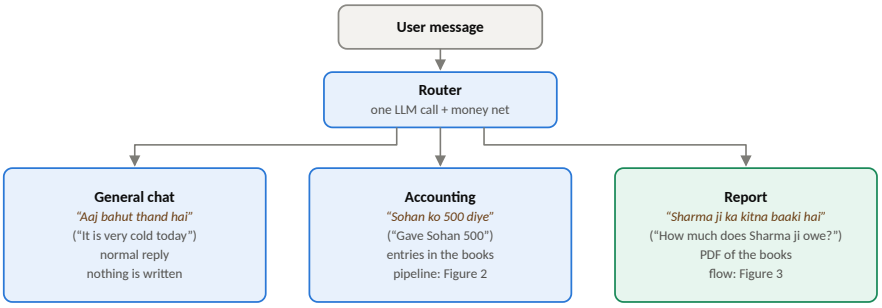



**Figure 1.** The three paths after the router. Blue boxes use the LLM; the report path (green) uses it only to read the request and returns a PDF.

The accounting flow is a LangGraph sub-graph (Figure 2). Each message is split into actions, the actions are grouped by who or what they concern, and each group is sent down one of three routes in parallel. The routes join again before the system chooses tools and fills their arguments. Nothing is written until the user confirms.

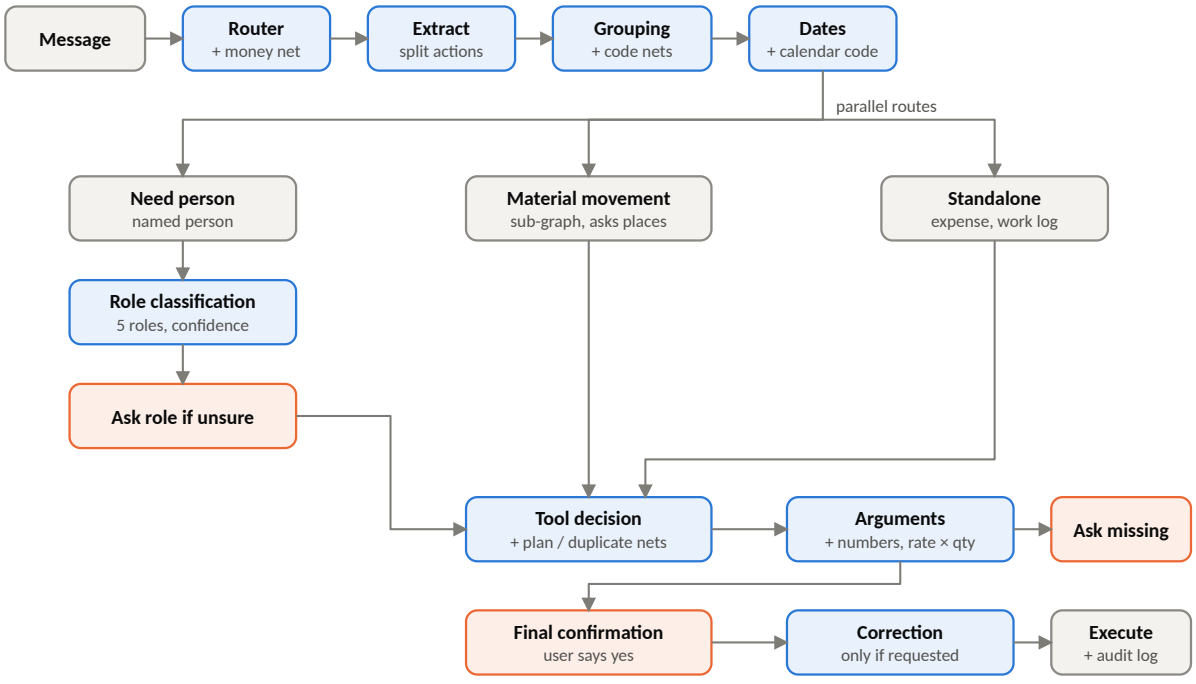



**Figure 2.** Accounting pipeline. Blue boxes call the LLM; every LLM step is followed by deterministic code nets. Orange boxes pause and ask the user.

## 3.2 The steps

| Step | What it does | Example |
|---|---|---|
| Router | Chat, report or accounting. A code net catches money statements the LLM sends to chat. | *"Papa galle se kuch paise le gaye the shaam ko"* "Papa took some money from the till in the evening" → accounting |
| Extract | Splits the message into single actions, each with a date. | *"Pappu ko kal 300 aur aaj 400 diye"* "Gave Pappu 300 yesterday and 400 today" → two payments |
| Grouping | Puts actions into groups: `NEED_PERSON`, `MATERIAL_MOVEMENT`, `STANDALONE` (routed in parallel) or `CORRECTION` (run after confirmation). | *"Gudam se dukaan 30 chaukhat bheji"* "Sent 30 door frames from the godown to the shop" → movement godown → shop |
| Dates | LLM reads the date words; code computes the calendar date and overrides doubtful cases. | "Kalanaur" is a village, not *kal* ("yesterday") |
| Role | Scores a person against five roles; asks if unsure. | *"Om Prakash ji se purani haveli 5 lakh mein khareedi"* "Bought an old mansion from Om Prakash ji for 5 lakh" → Om Prakash ji = property owner |
| Tool decision | Chooses tools; code nets block future plans and duplicates. | *"Pappu ko agle hafte 500 denge"* "Will give Pappu 500 next week" → no entry (a plan) |
| Arguments | Fills fields; code computes rate × quantity and number words. | *"Gudam se 2000 eent Sharma ji ke plot pe gayi, 6 ka rate"* "2000 bricks went from the godown to Sharma ji's plot, at 6 each" → 2000 × ₹6 = ₹12,000 |
| Confirm, correct, execute | User confirms; corrections are reversal entries, never edits; every write goes to an audit log. | |

## 3.3 The report flow

A report request goes through five short steps (Figure 3). The LLM is used only to read the request. Every number in the PDF (balances, totals, cash in hand) is computed by SQL queries and Python, so a report is exact whenever the entries are. Periods longer than a month are shown as a monthly summary plus full detail of the last month.

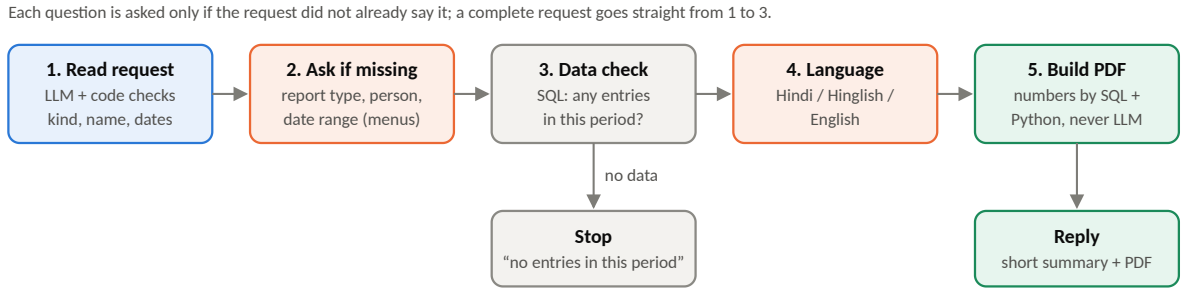



**Figure 3.** Report flow. Orange steps ask the owner a menu question, only when the request did not already answer it.

## 3.4 Why a hybrid of LLM and code

The LLM handles wording no rule could foresee. Code handles what must be exact and repeatable: arithmetic, dates, number words, and known business facts (a *gudam*, the godown, is our own place; money is never "material"). When a test failed, the first choice was a code net placed after the LLM, because the same input then always gives the same correction. Prompt changes were used only when the LLM lacked a concept, and every new prompt example was checked against all test statements for leakage (Section 4). Of 70 logged changes, 53 were code rules or code fixes, 8 paired a prompt change with a code rule, 1 restructured a prompt to cut cost, and 8 changed only the test harness (runner, scorer, test set; Appendix B).

## 3.5 What is general and what is configured

The core of the system does not depend on bricks or timber. Only a small configuration layer describes this particular business:

| General (any trading business) | Configured for this business |
|---|---|
| 18 tools: wages, attendance, sales, purchases, supplier deliveries, payments, expenses, stock moves, adjustments, family withdrawals, opening balances | Item list and spellings (*eent*, *shahteer*, *balli*, *chaukhat* = brick, wooden beam, pole, door frame) |
| Money vs goods; transaction separate from payment; append-only stock ledger; cash ledger | Place names (Kalanaur, Dhani, *gudam*, *dukaan*) and their types (site, storage, shop) |
| Asking a person's role when unsure; confirmation before any write; reversal-based corrections | Seed data: names of employees, customers, suppliers and family members |
| Hinglish number words, relative dates, grouping of actions | A few domain words (*malba*, *haveli todna* = debris, demolishing a mansion) |

The grouping prompt is written to be business-agnostic, and each of the seven entry bundles has a category of ten statements in the wording of other businesses (Section 5.2).

## 3.6 Data model

The PostgreSQL schema (SQLAlchemy, 27 tables) separates **transactions** (what was sold or delivered, at what rate) from **payments** (money only), so advances and part-payments work naturally. Stock is an append-only ledger: `Inventory` holds the current count and every change is logged in `Adjustment` with its source. A single `CashLedger` records all money in and out. Corrections never edit a row: the old row is marked *corrected* and a reversal row is added. The audit log stores the original sentence, the tool and its arguments.

## 3.7 Cost

The argument-extraction prompt was split by bundle, from about 14k to about 3k tokens, with 49 of 50 outputs identical in an ablation check. This cut tokens by about 79% and average time from 15.1 to 11.8 seconds per test.

Measured on the OpenAI usage dashboard during the test runs, 50 statements cost about US$0.20–0.29 with gpt-4o-mini, i.e. roughly half a US cent per statement. A shop with 30 entries a day (about 900 a month) would pay about US$3.60–5.20 a month, roughly ₹300–450 at ₹85–88 per dollar. Test statements also include the questions the system asks, so this is a fair estimate of real use.

# 4. Evaluation methodology

## 4.1 Test sets

| Set | Statements | Purpose |
|---|---|---|
| Development bundles: Employee, Customer, Supplier, Expense, Family, Stock, Purchase | 7 × 50 (+10 field tests) | Find and fix errors, one business area at a time |
| Development: Report, Correction, Router | 20 + 25 + 30 | Report requests, corrections of old entries, message routing |
| Holdout round 1 | 135 | First test on unseen data; later used to make fixes |
| Holdout round 2 (final) | 135 | Final test on unseen data; run once, no fixes after it |

Each of the seven entry bundles has five categories of ten statements: A daily, B tricky, C edge cases, D wording from other businesses (for example *“Gupta ji ko 10 kilo cheeni 450 mein di, udhaar”* (“Gave Gupta ji 10 kilo sugar for 450, on credit”)), E several actions in one statement. Report, Correction and Router have four categories each (for example, for Router: general chat, chat that sounds like business, an entry that sounds like chat, and report or entry). Each holdout round has 15 statements per entry bundle (8 in my father’s own style, 7 mixed: daily, tricky, incomplete, several actions, and statements that should create no entry) and 10 each for Report, Correction and Router.

## 4.2 Rules that keep the numbers honest

1. **Locked before running.** Every statement and its expected answer was fixed before the system saw it. Expected answers were not changed after a run, except for three documented changes, each logged with its reason (for example, a driver field was removed from two expectations because the movement tool has no such field).
2. **Leakage check.** Every new prompt example was compared with every test statement using difflib similarity. Anything above 0.69 was rewritten, so the prompt never contains a near-copy of a test.
3. **Full runs only.** A reported “after” number comes from one complete run of the bundle, with all tests run on the same code, and its run file is kept (Appendix C). Correction is the one exception, marked as “latest result per test”.
4. **Fresh database.** The test database is wiped and re-seeded before every statement. The real database is never touched.
5. **Holdout discipline.** Holdout statements were drafted after development ended, with the help of an AI assistant, and reviewed and locked by me before the run. Round 2 was run once, and no fix was made after it.

## 4.3 Scoring

Every expected entry is one **row**. A wrong extra entry also adds a row. Each row gets one label:

| Label | Meaning |
|---|---|
| Correct | Right tool and every key field right (type, amount, date, person; for goods also item, quantity, rate and place). If the test expects a question and the system asks for the missing value, the row is Correct. |
| Safe ask | The system stopped and asked when it did not need to. The user loses time, but the books stay right. |
| Silent wrong | A wrong entry, or an entry that should not exist, prepared for the user to confirm. This is the dangerous case. |
| Missed | No entry and no question: something that happened was not recorded. |

**Strict accuracy** = Correct / rows. **Safe rate** = (Correct + Safe ask) / rows. **Silent-error rate** = Silent wrong / rows. A test is **fully right** when all its rows are Correct.

All accuracy and error rates in this report are per **entry (row)**, not per statement: one statement can produce several entries, so 135 holdout statements gave 166 scored rows in round 2. The statement-level result is reported separately as “fully right”.

**Confidence intervals.** With 15 statements a single test changes a bundle by several points, so every rate is reported with a 95% Wilson interval [1]. Two results are treated as different only when their intervals do not overlap.

## 4.4 Test harness

A runner sends each statement through the real graph. When the system pauses to ask a role or a place type, the runner answers from the test file, so it does not hide errors. Any other question ends the test and is scored as asked. Report and correction tests stop at their first question and check what was understood (route, kind of report, name, dates). Router tests run the router step only.

# 5. Results

## 5.1 Development bundles: before and after fixes

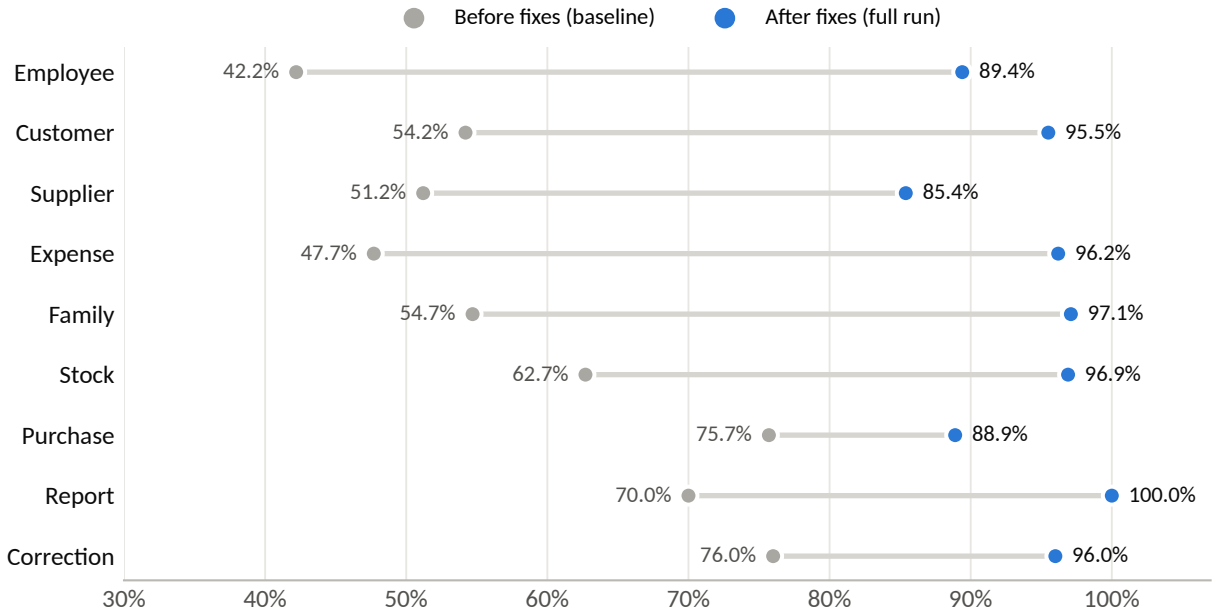



**Figure 4.** Strict accuracy per development bundle before fixes (baseline run) and after fixes (last full run). Router (100%, 30 tests) was run once with no fixes and is not shown.

| Bundle | Rows | Before | After | 95% CI (after) | Silent errors before → after |
|---|---|---|---|---|---|
| Employee | 66 | 42.2% | 89.4% | 79.7–94.8% | 23.4% → 7.6% |
| Customer | 67 | 54.2% | 95.5% | 87.6–98.5% | 16.7% → 1.5% |
| Supplier | 82 | 51.2% | 85.4% | 76.1–91.4% | 15.1% → 8.5% |
| Expense | 105 | 47.7% | 96.2% | 90.6–98.5% | 10.1% → 0.0% |
| Family | 69 | 54.7% | 97.1% | 90.0–99.2% | 16.0% → 1.4% |
| Stock | 64 | 62.7% | 96.9% | 89.3–99.1% | 6.0% → 0.0% |
| Purchase | 72 | 75.7% | 88.9% | 79.6–94.3% | 10.8% → 2.8% |
| Report | 20 | 70.0% | 100% | 83.9–100% | 25.0% → 0.0% |
| Correction* | 25 | 76.0% | 96.0% | 80.5–99.3% | 24.0% → 4.0% |
| Router | 30 | — | 100% | 88.6–100% | — → 0.0% |

Rows are the after-run counts (the before runs had slightly different counts because of extra entries). *Correction = latest result per test (full run plus a re-run of 9 tests), not one full run. Employee and Customer full runs were made on 5 October; later shared fixes were re-checked on about 40 selected tests, not by full re-runs.

These numbers show that the fixes work on the statements they were made for. They do not show how the system handles new wording, because every fix was made while looking at these tests. That is what the holdout rounds measure.

## 5.2 Wording from other businesses

Category D of the seven entry bundles (70 statements, such as a kirana sale of sugar, a dairy freezer repair or a tailor’s piece-rate work) checks whether the system handles

businesses other than the test bed. Strict accuracy on these rows rose from 39.4% before fixes (99 rows) to 94.7% after fixes (94 rows, 95% CI 88.1–97.7%), and silent errors fell from 17.2% to 3.2%. This is early evidence that the core is not tied to bricks and timber, but it is development data: fixes were made while looking at these statements too. A holdout from a second business is the proper test (Section 8).

## 5.3 Holdout: unseen data

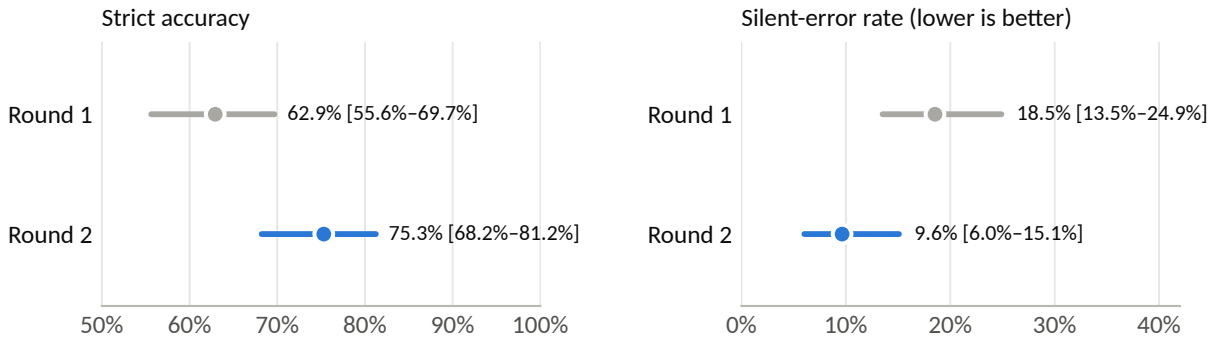



**Figure 5.** Overall strict accuracy and silent-error rate with 95% Wilson intervals. Round 1: 178 rows; round 2: 166 rows.

Round 1 (62.9%) was well below the development numbers. This is the usual sign that development fixes were partly fitted to their tests. Its failures fell into seven families (place names read as dates, missing expense words, plans recorded as entries, profile statements split apart, and others), which were fixed and checked on a 19-test regression set. Round 2 then used 135 new statements and was run once.

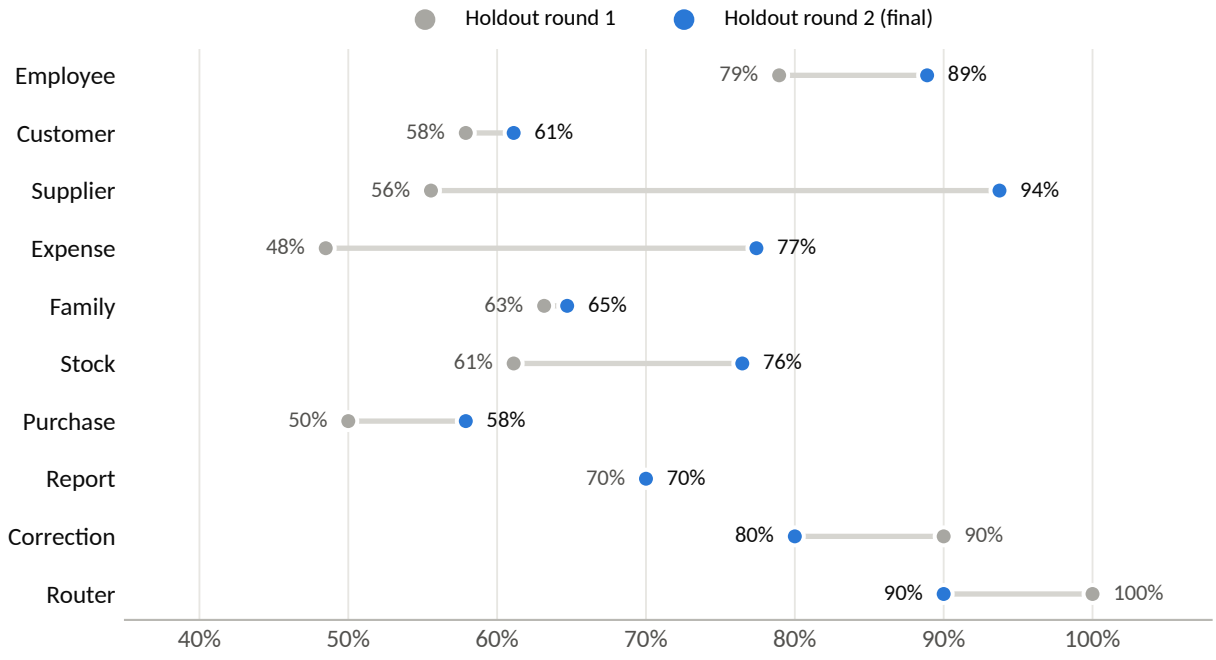



**Figure 6.** Strict accuracy per bundle, holdout round 1 vs round 2. Each bundle has 10–33 rows, so single-bundle differences are within noise (Table below).

| Bundle | R1 entry accuracy | R2 entry accuracy | R2 95% CI | R2 silent | Fully right R1 → R2 |
|---|---|---|---|---|---|
| Employee | 78.9% | 88.9% | 67.2–96.9% | 0.0% | 12/15 → 13/15 |
| Customer | 57.9% | 61.1% | 38.6–79.7% | 11.1% | 9/15 → 8/15 |
| Supplier | 55.6% | 93.8% | 71.7–98.9% | 6.3% | 9/15 → 14/15 |
| Expense | 48.5% | 77.4% | 60.2–88.6% | 3.2% | 6/15 → 12/15 |
| Family | 63.2% | 64.7% | 41.3–82.7% | 11.8% | 10/15 → 10/15 |
| Stock | 61.1% | 76.5% | 52.7–90.4% | 5.9% | 10/15 → 12/15 |
| Purchase | 50.0% | 57.9% | 36.3–76.9% | 15.8% | 8/15 → 10/15 |
| Report | 70.0% | 70.0% | 39.7–89.2% | 30.0% | 7/10 → 7/10 |
| Correction | 90.0% | 80.0% | 49.0–94.3% | 20.0% | 9/10 → 8/10 |
| Router | 100% | 90.0% | 59.6–98.2% | 10.0% | 10/10 → 9/10 |
| **All** | **62.9%** | **75.3%** | **68.2–81.2%** | **9.6%** | **90/135 → 103/135** |

## 5.4 By statement type

| Entry bundles only | R1 entry accuracy | R2 entry accuracy | R2 95% CI | R1 silent | R2 silent |
|---|---|---|---|---|---|
| Papa-style (8 per bundle) | 57.3% | 75.7% | 64.5–84.2% | 22.7% | 5.7% |
| Mixed (7 per bundle) | 58.9% | 72.7% | 61.0–82.0% | 16.4% | 9.1% |
| **All entry bundles** | **58.1%** | **74.3%** | **66.3–80.9%** | **19.6%** | **7.4%** |

## 5.5 What the results say

- **The improvement from round 1 to round 2 is unlikely to be chance.** For the entry bundles the accuracy intervals (50.1–65.8% and 66.3–80.9%) do not overlap. The two rounds used different statements, so part of the change could come from round 2 being easier.
- **Silent errors roughly halved**, from 18.5% to 9.6% overall. The overall intervals still overlap (13.5–24.9% and 6.0–15.1%), but for the entry bundles they do not (14.0–26.7% and 4.0–13.0%), and on the owner's own style of speech the rate fell from 22.7% to 5.7%. When the system is wrong it now more often stops or asks instead of writing a wrong entry.
- **About 75% of entries is the realistic estimate** on new wording, well below the 85–100% development numbers. The gap measures how much of the development gain came from fitting specific tests.
- **Weakest areas:** Purchase, Customer and Family (58–65%). Most of their errors share the general causes in Section 6, not bundle-specific ones.
- Per-bundle round-2 differences, such as Router 100% → 90% or Correction 90% → 80%, are one test each and lie inside the intervals.

# 6. Error analysis

All 32 round-2 statements that were not fully right were reviewed, and the main cases were traced through the step logs. Most errors fall into a few families. Each family is shown with a real case and the step where it failed.

### 1. A short name matched inside a longer name Silent wrong

*“Mange Ram ko makaan ke 50000 diye”* (“Paid Mange Ram 50,000 for the house”) (PUR-11; same in PUR-13, PUR-14)
**Expected:** purchase payment ₹50,000 to the seller Mange Ram. **Got:** wage payment ₹50,000 to employee “Ram”.
**Cause:** the grouping LLM read “mange” as the verb *maange* (“asked”) and kept only “Ram”, which is an employee. Three code nets that look up known names also match “Ram” inside “Mange Ram”. This is the most serious remaining error: a large amount written to the wrong account.

### 2. A place read as a person Silent wrong

*“JCB ka 3 ghante ka 2700 diya Kalanaur site pe”* (“Paid 2700 for 3 hours of JCB at the Kalanaur site”) (EXP-11)
**Expected:** machine-hire expense ₹2,700. **Got:** wage of ₹2,700 to an employee named “Kalanaur”.
**Cause:** nothing tells the grouping step that site names are never people.

### 3. “le gaya”: goods or money? Missed

*“Naresh aaj aaya aur 600 le gaya”* (“Naresh came today and took 600”) (EMP-07): attendance was recorded, but the ₹600 was lost. *“Dalbir 300 eent le gaya”* (“Dalbir took 300 bricks”) (CUS-07): no sale was recorded.
**Cause:** the grouping prompt lists “le gaya” (“took”) as a sign that goods moved. For EMP-07 the LLM built a goods movement with no goods; a code net then turned it into a nameless “payment” and dropped Naresh’s name, so no tool could be chosen. The verb is neutral. What decides it is the object: a number alone means money, an item word means goods.

### 4. An amount separated from its owner Missed

*“Ram ne gudam mein darwaze chhaante, 700 bante hain”* (“Ram sorted doors at the godown, that comes to 700”) (EMP-03)
**Cause:** extraction split off “700 bante hain” (“that comes to 700”) with no name. The grouping step sees only the fragments, not the original sentence, so it cannot tell whose 700 it is. The almost identical EMP-09 passed because the LLM kept it as one action. The step is not consistent.

### 5. Plans, home and business Silent wrong Missed

*“Diwali pe ghar ki safai karwani hai, 3000 lagenge”* (“The house has to be cleaned for Diwali, it will cost 3000”) (FAM-08) was recorded as a ₹3,000 family expense, although it is a future plan. *“Ghar ki gas khatam thi, nayi tanki 1050 mein bharwayi”* (“The home gas ran out, got a new cylinder filled for 1050”) (FAM-05) was recorded as business fuel, not a home expense. Stock shortages (*“12 darwaze kam pade”* (“12 doors were short”), STK-05) were not recorded as adjustments.

**6. Report and correction requests** Silent wrong

*“Sohan ki is mahine ki dihadi aur payment ki report bhejo”* (“Send a report of Sohan’s wages and payments this month”) (RPT-01) produced a general work-log report without Sohan’s name. *“Kal Sohan ki mazdoori do baar chadh gayi”* (“Yesterday Sohan’s wage got entered twice”) (COR-01) was not recognised as a correction, because “entered twice” is not among the correction signals.

## 6.1 A pattern across the families

The families share three structural causes, not missing words:

1. **Lost context.** The grouping step sees fragments, not the original sentence (families 1 and 4).
2. **Rules that overwrite.** Some code nets replace information instead of adding to it, for example the net that dropped Naresh’s name and a correction net that can replace every group in a message. Name matching does not prefer the longest name.
3. **Examples instead of principles.** The 1,600-line grouping prompt teaches by example and contains contradictions (it says there are four group types, and elsewhere “only three”). New wording falls between the examples.

A full read of the grouping code and prompt confirmed these points. They are the basis of the roadmap in Section 8.

# 7. Limitations

- **Realistic, but not recorded, statements.** The statements follow the way my father actually speaks: his words, names, places and way of saying numbers. By my own estimate, about 35 of every 50 statements in the development bundles are phrased exactly as he would say them; the rest are deliberately harder variations. In the holdout rounds, 8 of every 15 statements per bundle were written in his style by design. All sets were drafted with the help of an AI assistant and checked and locked by me before they were run. They are not yet recordings from live use, which will be noisier: speech-to-text errors, unfinished sentences, new names.
- **One business.** All testing was on one business. The core is general (Section 3), and the development set includes wording from other businesses, but transfer to a second business has not been measured on unseen data.
- **Same people behind system and tests.** I built the system and approved every test, and the same AI assistant helped with both. Locking the tests, the leakage check and the holdout rounds reduce this bias but do not remove it. An independent test set from another user would be stronger.
- **Small samples.** 10–33 rows per bundle in each holdout round give intervals about 27–50 points wide. Only the overall numbers are precise enough for firm claims.
- **Hinglish only.** Only Hinglish was tested. Hindi (Devanagari) and English input are future work.
- **Partial coverage of the flow.** Report tests check only the understanding of the request, not the PDF or its totals. Correction tests stop at the first question of the correction menu.
- **Non-determinism.** Even at temperature 0 the LLM sometimes splits or groups the same sentence differently (EMP-03 vs EMP-09). A single run per test does not measure this variance.

# 8. Conclusion and future work

Hisab-Kitab AI turns a small business owner's spoken Hinglish notes into structured, confirmed accounting entries. On 135 statements it had never seen (166 entries), 75% of entries are strictly correct and 103 statements are fully right, and it writes a silent wrong entry for under 10% of entries, about half the rate of the first holdout round. The development scores of 85–100% are real for those tests, but they overstate performance on new wording. Measuring that gap was the main purpose of the holdout rounds.

The most useful lesson is about **how** to fix. Fixes that taught a specific word helped the tests they were made for and transferred only partly. The errors that remain point to general causes, so the next version should change principles, not add more words:

| # | Change | Addresses |
|---|---|---|
| 1 | Give the grouping step the original sentence as well as the fragments. | Lost owner of an amount; shortened names |
| 2 | Rewrite the grouping prompt around five decision principles: what moved (goods or money); copy names exactly; every amount has an owner; when it happened (now, a past mistake, or a plan); place or person. | New wording falling between examples; contradictions |
| 3 | Code nets may add or rename but never discard a name or a group; the longest known name wins; merge duplicate groups once more at the end. | Mange Ram, Naresh, over-broad correction net |
| 4 | When unsure, ask: *"ye 600 kiske hain?"* ("whose 600 is this?") instead of dropping the amount. | Turns Missed into a question |
| 5 | Log every user correction and answered question in real use, and review them weekly as the next test set. | Real wording; replaces written statements |
| 6 | A holdout of 30–50 statements from a second business (for example a kirana or hardware shop), with only its configuration changed. | Measures transfer beyond the test bed |
| 7 | Pending Purchase fixes; request timeout; repeated runs to measure variance. | Open items from this phase |

**Next steps beyond this report.** Hisab-Kitab AI is not meant for one shop. It is meant for every small business owner like my father, and it only becomes valuable when many of them use it.

1. **Languages.** Test Hindi (Devanagari) and English input with the same locked-test method.
2. **Deploy.** Once the app is ready, give it to the local business owners I know (kirana, hardware, dairy and building-material shops, contractors), starting with my father's business as the first user.
3. **Improve from real use.** With each owner's consent, log the corrections they make and the questions the system asks, turn them into new test sets, and update the system, so that it keeps getting better for every kind of business, not only the first one.

Changes 1–3 touch every bundle, so they need full re-runs of all development sets and a fresh holdout round. They were deliberately postponed past this report so that the reported numbers stay tied to the system that produced them.

# References


1. E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
2. OpenAI. GPT-4o mini: advancing cost-efficient intelligence. Product announcement, July 2024.
3. S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram, M. Choudhury. GLUECoS: An evaluation benchmark for code-switched NLP. In *Proc. ACL*, 2020.
4. LangChain. LangGraph documentation: graphs, sub-graphs, human-in-the-loop and persistence. langchain-ai.github.io/langgraph.
5. M. T. Ribeiro, T. Wu, C. Guestrin, S. Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proc. ACL*, 2020.
6. M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, T. Gebru. Model cards for model reporting. In *Proc. FAT**, 2019.
7. S. Kapoor, A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.

# Appendix A — Example test statements

Examples from holdout round 2 with the expected entry and the result. Each Hinglish statement is followed by an English translation. The companion workbook lists all 695 statements with expected and actual output.

| ID | Statement | Expected | Result |
|---|---|---|---|
| EMP-04 | *Kitanu aaj kaam pe aaya, tractor bhi usi ne chalaya*<br>Kitanu came to work today and also drove the tractor | attendance P, driving P | Correct |
| EMP-14 | *pappu kal aadha din aaya tha, aaj poora*<br>Pappu came for half a day yesterday, full day today | half day yesterday, full day today | Correct |
| CUS-05 | *Suresh Traders 5 shahteer le gaye 10000 ke, 4000 de gaye*<br>Suresh Traders took 5 wooden roof beams worth 10,000 and paid 4,000 | sale 5 wooden roof beams ₹10,000 + payment ₹4,000 | Correct |
| CUS-14 | *dalbir ko sava sau eent saade paanch ke rate se*<br>125 bricks to Dalbir at a rate of 5.5 | sale 125 bricks × ₹5.5 = ₹687.5 | Silent wrong 750 × 5 |
| SUP-03 | *SBI se 2500 eent dukaan pe utri 17500 ki, Kitanu laya*<br>2500 bricks from SBI unloaded at the shop, worth 17,500; Kitanu brought them | supplier delivery to shop, driver Kitanu | Correct |
| SUP-15 | *Hanuman wale ki gaadi aayi thi eent lekar*<br>Hanuman's vehicle came with bricks | ask how many bricks | Silent wrong recorded without asking |
| EXP-15 | *Diesel 1500 ka aur tyre puncture 200 ka*<br>Diesel for 1500 and a tyre puncture for 200 | two expenses + two payments | Correct |
| FAM-02 | *Bhai galle se 2500 le gaya bike ki kisht ke liye*<br>Brother took 2500 from the till for the bike instalment | family withdrawal ₹2,500, purpose | Correct |
| STK-06 | *dhani wali site se dedh sau eent gudm aayi*<br>150 bricks came to the godown from the Dhani site | movement 150 bricks site → godown | Correct |
| PUR-01 | *Om Prakash ji se purani haveli 5 lakh mein khareedi*<br>Bought an old mansion from Om Prakash ji for 5 lakh | purchase ₹5,00,000, seller Om Prakash ji | Correct |
| PUR-11 | *Mange Ram ko makaan ke 50000 diye*<br>Paid Mange Ram 50,000 for the house | purchase payment ₹50,000 | Silent wrong wage to "Ram" |
| RPT-08 | *Suresh Traders ki saari entry nikal do*<br>Show all entries of Suresh Traders | person report, Suresh Traders | Correct |
| COR-04 | *Kitanu ko aaj 300 pakdaye, aur kal ki uski absent wali entry hata do*<br>Gave Kitanu 300 today, and delete his absent entry of yesterday | new payment + correction (Kitanu, yesterday) | Correct |
| RTR-03 | *Sohan ki maa ab theek hain kya?*<br>Is Sohan's mother well now? | general chat (not an entry) | Correct |
| RTR-06 | *Sharma ji ka beta 1500 de kar gaya abhi*<br>Sharma ji's son just came and paid 1500 | accounting (payment) | Correct |

# Appendix B — Fix log summary

Every change was logged with its date, bundle, type, file and the tests it fixed (workbook sheet “Changelog”, 70 entries).

| Type of change | Count |
|---|---|
| Code rule after the LLM | 50 |
| Prompt change together with a code rule | 8 |
| Code fix (crash) / code rule with data-model change | 3 |
| Prompt structure (cost) | 1 |
| Test harness only: runner (4), scorer (3), test set (1) | 8 |
| **Total** | **70** |

## Representative fixes

| Area | Fix |
|---|---|
| Numbers | Number words to digits in three languages (*dedh*, *saade*, *sava*, *paune*, *sau*, *hazaar*, *lakh*, *5k* = 1.5, +0.5, +0.25, −0.25, hundred, thousand, lakh, 5000), with 34 unit tests; rate × quantity = total. |
| Dates | Weekday = last such day; *“agle hafte ki mazdoori”* (“next week’s wages”) is a purpose, not a date; a statement with no date word is dated today, so “Kalanaur” no longer means “yesterday”. |
| Money vs goods | Money given to a person is never a material movement; expense words (diesel, bill, rent) are never goods. |
| Expenses | A paid expense becomes expense + payment (only the payment moves cash); *“baad mein denge”* (“will pay later”) = expense only. |
| Family | The till (*galla*) is never a person; relation words (Papa, Mummy, Bhai) act as names. |
| Plans | Future or planned actions (*denge*, *bhejni hai*, *padenge*, *uthwana hai* = will give, has to be sent, will have to, has to be lifted) create no entry. |
| Corrections | Old-entry talk split off by extraction is joined back to the correction, not recorded as a new entry. |
| Holdout round 1 | Seven groups (H1–H7): purchases of machines and property, driver and direction, new expense words, dates, plans and requests, profile statements, report names; plus three regression fixes found on a 19-test check. |

# Appendix C — Proof files and test dates

All tests ran between 4 and 8 October 2026 with gpt-4o-mini at temperature 0, Hinglish input. Every reported number can be recomputed from these run files (`test/results/<bundle>/`) with the scoring scripts.

| Number | Run file |
|---|---|
| Employee before | `v1 run of 4 Oct (Hinglish)` |
| Employee after | `emp_hinglish_2026-10-05_1349.json` |
| Customer before / after | `cust_hinglish_2026-10-05_1757.json / cust_hinglish_2026-10-05_2214.json` |
| Supplier before / after | `supp_hinglish_2026-10-06_0035.json / supp_hinglish_2026-10-06_1527.json` |
| Expense before / after | `exp_hinglish_2026-10-06_1646.json / exp_hinglish_2026-10-06_1742.json` |
| Family before / after | `fam_hinglish_2026-10-06_1823.json / fam_hinglish_2026-10-07_1215.json` |
| Stock before / after | `stk_hinglish_2026-10-08_0619.json / stk_hinglish_2026-10-08_0707.json` |
| Purchase before / after | `pur_hinglish_2026-10-08_0800.json / pur_hinglish_2026-10-08_1317.json` |
| Report before / after | `rpt_hinglish_2026-10-08_1058.json / rpt_hinglish_2026-10-08_1116.json` |
| Correction before / after | `cor_hinglish_2026-10-08_1128.json / + cor_hinglish_2026-10-08_1157.json (re-run of 9)` |
| Router | `rtr_hinglish_2026-10-08_1227.json` |
| Holdout round 1 | `fin_hinglish_2026-10-08_1440.json` |
| Holdout round 2 (final) | `fin2_hinglish_2026-10-08_1651.json` |

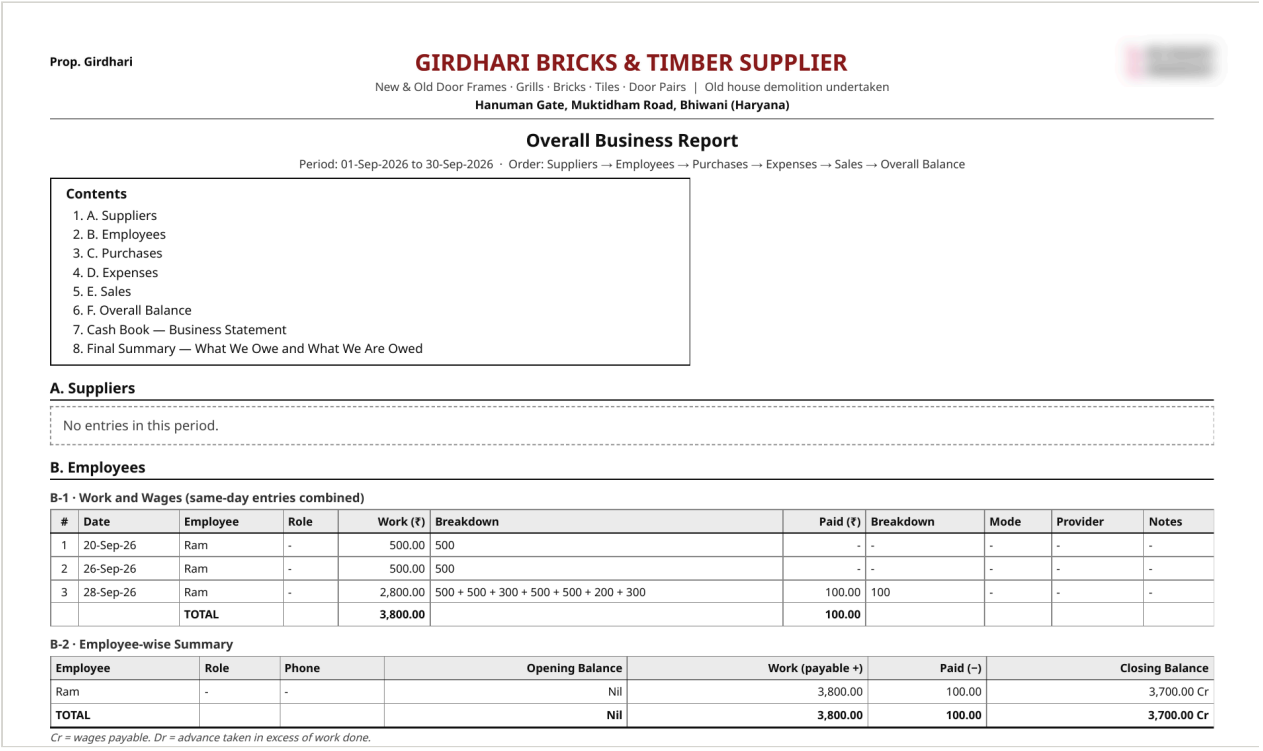
# Appendix D — Scoring rules in detail

- **Key fields** checked per tool: type, amount, date and person for all money entries; item, quantity, rate, source, destination and driver (where the tool has the field) for goods; kind, name and date range for reports; name and date for corrections; route for router tests.
- **Names:** a misspelt name at least 80% similar to the expected one counts as the same (the system cannot know the spelling of a name that is not in the database). A report or correction name must match on its first word.
- **Units:** 2 ton = 2000 kg.
- **Expense payments:** an expense payment equal to an expense in the same statement is not an extra error (older test files were written before the expense + payment model).
- **Questions:** when a test expects a question and the system asks for the missing value, or shows the *“samajh nahi aaya, kisko/kisse”* (“didn’t understand, to whom / from whom?”) message, the row is Correct (or Safe ask for the message).
- **Runner answers** only role and place-type questions, from the test file. A name that is not in the test’s roles stops the test, so wrong names are not hidden.
- **Debatable rows kept as scored:** two round-2 rows could be argued either way, and both were counted as wrong: *“pichle paanch din”* (“the last five days”) (does the window include today?) and a missing quantity of 1 for *“ek purani almari”* (“one old cupboard”). Counting both as correct would raise round 2 from 75.3% to 76.5%.
- **Wilson interval** for k correct out of n rows, z = 1.96: centre $(\hat{p} + z^2/2n)/(1 + z^2/n)$, half-width $z \cdot \sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)}/(1 + z^2/n)$.

# Appendix E — Sample report output

Hisab-Kitab AI produces seven kinds of report (overall business, a person's ledger, purchases, expenses, cash flow, work log and material movement), each as a PDF in Hindi, Hinglish or English, as the owner chooses. The **overall business report** has eight sections: suppliers, employees, purchases, expenses, sales, overall balance, cash book, and a final summary of what the business owes and is owed. Below is the first page of the overall report for September 2026, showing the employee section, in all three languages. The same numbers appear in each version; only the language changes.

Generated by the app from development data. The letterhead is the real business, shown with the owner's first name only and phone numbers hidden. The PDF layout and totals were not part of the scored tests (Section 7).



Prop. Girdhari

**GIRDHARI BRICKS & TIMBER SUPPLIER**

New & Old Door Frames · Grills · Bricks · Tiles · Door Pairs | Old house demolition undertaken

**Hanuman Gate, Muktidham Road, Bhiwani (Haryana)**

**Overall Business Report**

Period: 01-Sep-2026 to 30-Sep-2026 · Order: Suppliers → Employees → Purchases → Expenses → Sales → Overall Balance

**Contents**

1. A. Suppliers
2. B. Employees
3. C. Purchases
4. D. Expenses
5. E. Sales
6. F. Overall Balance
7. Cash Book — Business Statement
8. Final Summary — What We Owe and What We Are Owed

**A. Suppliers**

No entries in this period.

**B. Employees**

**B-1 · Work and Wages (same-day entries combined)**

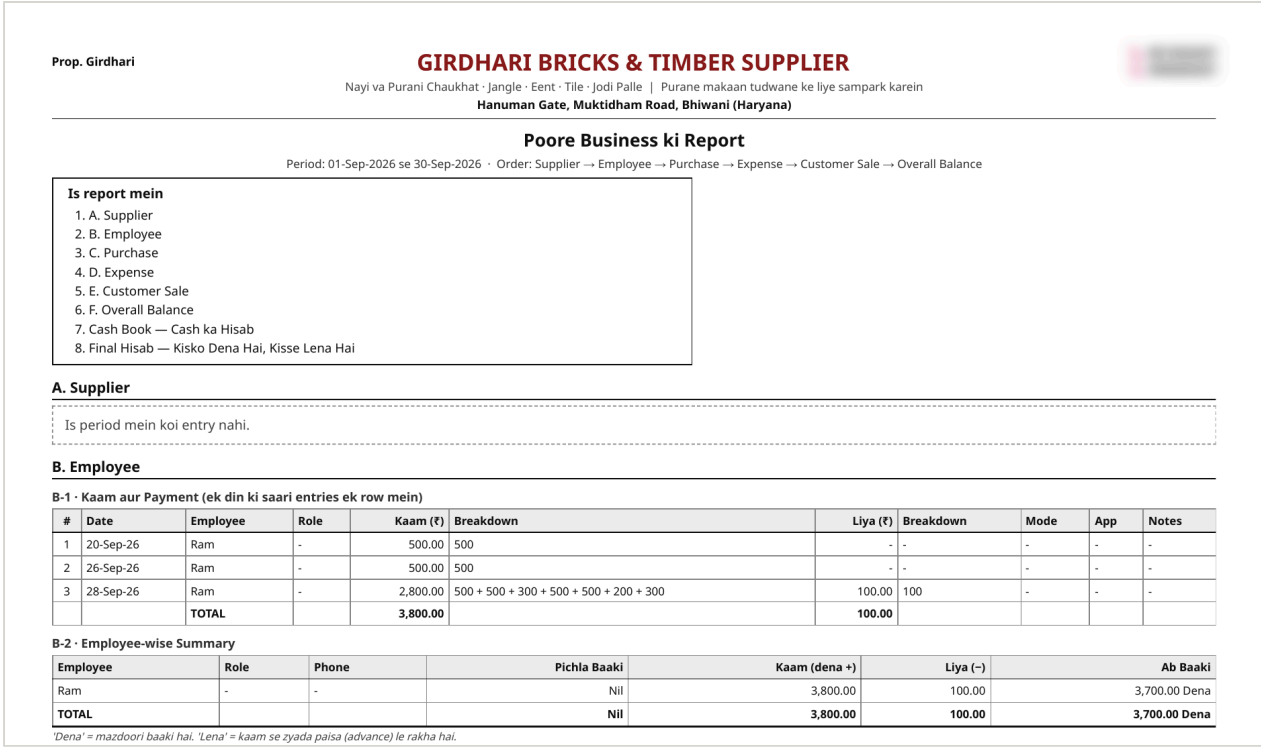
| # | Date | Employee | Role | Work (₹) | Breakdown | Paid (₹) | Breakdown | Mode | Provider | Notes |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 20-Sep-26 | Ram | - | 500.00 | 500 | - | - | - | - | - |
| 2 | 26-Sep-26 | Ram | - | 500.00 | 500 | - | - | - | - | - |
| 3 | 28-Sep-26 | Ram | - | 2,800.00 | 500 + 500 + 300 + 500 + 500 + 200 + 300 | 100.00 | 100 | - | - | - |
| | | **TOTAL** | | **3,800.00** | | **100.00** | | | | |

**B-2 · Employee-wise Summary**

| Employee | Role | Phone | Opening Balance | Work (payable +) | Paid (−) | Closing Balance |
|---|---|---|---|---|---|---|
| Ram | - | - | Nil | 3,800.00 | 100.00 | 3,700.00 Cr |
| **TOTAL** | | | **Nil** | **3,800.00** | **100.00** | **3,700.00 Cr** |

*Cr = wages payable. Dr = advance taken in excess of work done.*

**Figure E1.** English.



Prop. Girdhari

GIRDHARI BRICKS & TIMBER SUPPLIER

Nayi va Purani Chaukhat · Jangle · Eent · Tile · Jodi Palle | Purane makaan tudwane ke liye sampark karein

Hanuman Gate, Muktidham Road, Bhiwani (Haryana)

Poore Business ki Report

Period: 01-Sep-2026 se 30-Sep-2026 · Order: Supplier → Employee → Purchase → Expense → Customer Sale → Overall Balance

Is report mein

1. A. Supplier
2. B. Employee
3. C. Purchase
4. D. Expense
5. E. Customer Sale
6. F. Overall Balance
7. Cash Book — Cash ka Hisab
8. Final Hisab — Kisko Dena Hai, Kisse Lena Hai

A. Supplier

Is period mein koi entry nahi.

B. Employee

B-1 · Kaam aur Payment (ek din ki saari entries ek row mein)

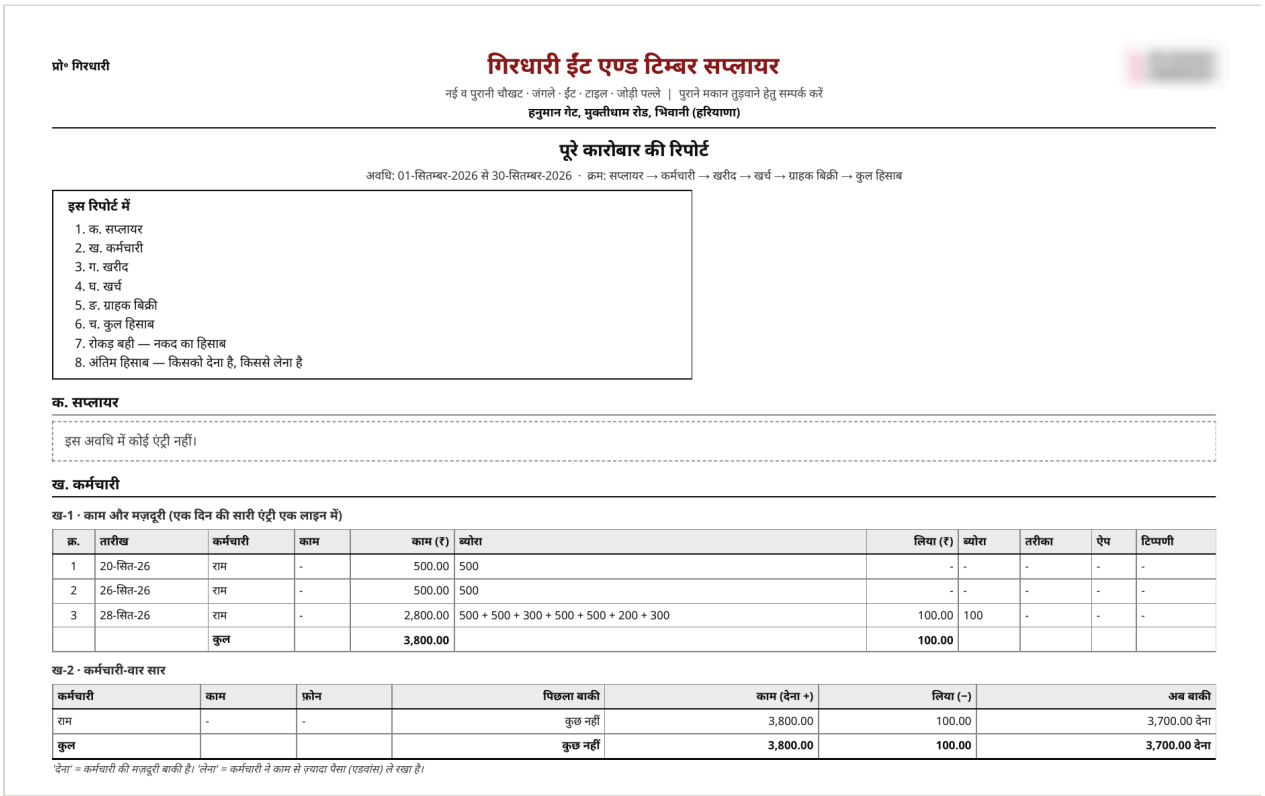
| # | Date | Employee | Role | Kaam (₹) | Breakdown | Liya (₹) | Breakdown | Mode | App | Notes |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 20-Sep-26 | Ram | - | 500.00 | 500 | - | - | - | - | - |
| 2 | 26-Sep-26 | Ram | - | 500.00 | 500 | - | - | - | - | - |
| 3 | 28-Sep-26 | Ram | - | 2,800.00 | 500 + 500 + 300 + 500 + 500 + 200 + 300 | 100.00 | 100 | - | - | - |
| | | TOTAL | | 3,800.00 | | 100.00 | | | | |

B-2 · Employee-wise Summary

| Employee | Role | Phone | Pichla Baaki | Kaam (dena +) | Liya (−) | Ab Baaki |
|---|---|---|---|---|---|---|
| Ram | - | - | Nil | 3,800.00 | 100.00 | 3,700.00 Dena |
| TOTAL | | | Nil | 3,800.00 | 100.00 | 3,700.00 Dena |

'Dena' = mazdoori baaki hai. 'Lena' = kaam se zyada paisa (advance) le rakha hai.

**Figure E2.** Hinglish.



प्रो॰ गिरधारी

गिरधारी ईंट एण्ड टिम्बर सप्लायर

नई व पुरानी चौखट · जंगले · ईंट · टाइल · जोड़ी पल्ले | पुराने मकान तुड़वाने हेतु सम्पर्क करें

हनुमान गेट, मुक्तीधाम रोड, भिवानी (हरियाणा)

पूरे कारोबार की रिपोर्ट

अवधि: 01-सितम्बर-2026 से 30-सितम्बर-2026 · क्रम: सप्लायर → कर्मचारी → खरीद → खर्च → ग्राहक बिक्री → कुल हिसाब

इस रिपोर्ट में

1. क. सप्लायर
2. ख. कर्मचारी
3. ग. खरीद
4. घ. खर्च
5. ङ. ग्राहक बिक्री
6. च. कुल हिसाब
7. रोकड़ बही — नकद का हिसाब
8. अंतिम हिसाब — किसको देना है, किससे लेना है

क. सप्लायर

इस अवधि में कोई एंट्री नहीं।

ख. कर्मचारी

ख-1 · काम और मज़दूरी (एक दिन की सारी एंट्री एक लाइन में)

| क्र. | तारीख | कर्मचारी | काम | काम (₹) | ब्योरा | लिया (₹) | ब्योरा | तरीका | ऐप | टिप्पणी |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 20-सित-26 | राम | - | 500.00 | 500 | - | - | - | - | - |
| 2 | 26-सित-26 | राम | - | 500.00 | 500 | - | - | - | - | - |
| 3 | 28-सित-26 | राम | - | 2,800.00 | 500 + 500 + 300 + 500 + 500 + 200 + 300 | 100.00 | 100 | - | - | - |
| | | कुल | | 3,800.00 | | 100.00 | | | | |

ख-2 · कर्मचारी-वार सार

| कर्मचारी | काम | फ़ोन | पिछला बाकी | काम (देना +) | लिया (−) | अब बाकी |
|---|---|---|---|---|---|---|
| राम | - | - | कुछ नहीं | 3,800.00 | 100.00 | 3,700.00 देना |
| कुल | | | कुछ नहीं | 3,800.00 | 100.00 | 3,700.00 देना |

'देना' = कर्मचारी की मज़दूरी बाकी है। 'लेना' = कर्मचारी ने काम से ज़्यादा पैसा (एडवांस) ले रखा है।

**Figure E3.** Hindi (Devanagari).